

② 事务的封锁请求可以随着事务的执行而动态地决定, 很难事先确定每一个事务要封锁哪些对象, 因此也就很难按规定的顺序去施加封锁。

可见, 在操作系统中广为采用的预防死锁的策略并不太适合数据库的特点, 因此数据库管理系统在解决死锁的问题上普遍采用的是诊断并解除死锁的方法。

2. 死锁的诊断与解除

数据库系统中诊断死锁的方法与操作系统类似, 一般使用超时法或事务等待图法。

(1) 超时法

如果一个事务的等待时间超过了规定的时限, 就认为发生了死锁。超时法实现简单, 但其不足也很明显, 一是有可能误判死锁, 如事务因为其他原因而使等待时间超过时限, 系统会误认为发生了死锁; 二是时限若设置得太长, 死锁发生后不能及时发现。

(2) 事务等待图法

事务等待图是一个有向图 $G=(T,U)$ 。其中 T 为结点的集合, 每个结点表示正在运行的事务; U 为边的集合, 每条边表示事务等待的情况。若 T_1 等待 T_2 , 则在 T_1 、 T_2 之间画一条有向边, 从 T_1 指向 T_2 , 如图 12.7 所示。

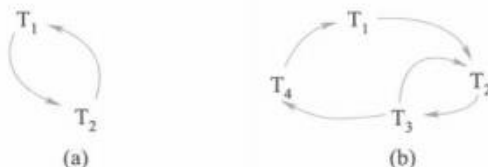


图 12.7 事务等待图示例

事务等待图动态地反映了所有事务的等待情况。并发控制子系统周期性地(比如每隔数秒)生成事务等待图并进行检测, 如果发现图中存在回路, 则表示系统中出现了死锁。

图 12.7(a)表示事务 T_1 等待 T_2 , T_2 又等待 T_1 , 产生了死锁。图 12.7(b)表示事务 T_1 等待 T_2 , T_2 等待 T_3 , T_3 等待 T_4 , T_4 又等待 T_1 , 产生了死锁。

当然, 死锁的情况可以多种多样。例如, 图 12.7(b)中事务 T_3 可能还等待 T_2 , 在大回路中又有小的回路。这些情况人们都已做了很深入的研究。

数据库管理系统的并发控制子系统一旦检测到系统中存在死锁, 就要设法解除。通常采用的方法是选择一个处理死锁代价最小的事务, 将其撤销, 释放此事务持有的所有锁, 使其他事务得以继续运行下去。当然, 对撤销的事务所执行的数据修改操作必须加以恢复。

12.6 并发调度的可串行性

数据库管理系统对并发事务不同的调度可能会产生不同的结果, 那么什么样的调度是正确的呢? 显然, 串行调度是正确的。执行结果等价于串行调度的调度也是正确的。这样的调度称为可串行化调度。

12.6.1 可串行化调度

多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同，称这种调度策略为可串行化调度。

可串行性是并发事务正确调度的准则。按这个准则规定，一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度。

[例 12.2] 现有两个事务，分别包含下列操作：

事务 T_1 ：读 B ； $A=B+1$ ；写回 A ；

事务 T_2 ：读 A ； $B=A+1$ ；写回 B 。

假设 A 、 B 的初值均为 2。按 $T_1 \rightarrow T_2$ 次序执行结果为 $A=3$ ， $B=4$ ；按 $T_2 \rightarrow T_1$ 次序执行结果为 $B=3$ ， $A=4$ 。

图 12.8 给出了对这两个事务不同的调度策略。其中，图 12.8(a) 和图 12.8(b) 为两种不同的串行调度策略，虽然执行结果不同，但它们都是正确的调度。图 12.8(c) 的执行结果与图 12.8(a)(b) 的结果都不同，所以是错误的调度。图 12.8(d) 的执行结果与图 12.8(a) 串行调度的执行结果相同，所以是正确的调度。

T_1	T_2	T_1	T_2	T_1	T_2	T_1	T_2
SLOCK B			SLOCK A	SLOCK B		SLOCK B	
$Y=R(B)=2$			$X=R(A)=2$	$Y=R(B)=2$		$Y=R(B)=2$	
UNLOCK B			UNLOCK A		SLOCK A	UNLOCK B	
XLOCK A			XLOCK B		$X=R(A)=2$	XLOCK A	
$A=Y+1=3$			$B=X+1=3$	UNLOCK B			SLOCK A
$W(A)$			$W(B)$		UNLOCK A	$A=Y+1=3$	等待
UNLOCK A			UNLOCK B	XLOCK A		$W(A)$	等待
	SLOCK A	SLOCK B		$A=Y+1=3$		UNLOCK A	等待
	$X=R(A)=3$	$Y=R(B)=3$		$W(A)$			$X=R(A)=3$
	UNLOCK A	UNLOCK B			XLOCK B		UNLOCK A
	XLOCK B	XLOCK A			$B=X+1=3$		XLOCK B
	$B=X+1=4$	$A=Y+1=4$			$W(B)$		$B=X+1=4$
	$W(B)$	$W(A)$		UNLOCK A			$W(B)$
	UNLOCK B	UNLOCK A			UNLOCK B		UNLOCK B

(a) 串行调度1

(b) 串行调度2

(c) 不可串行化的调度

(d) 可串行化的调度

图 12.8 并发事务的不同调度

12.6.2 冲突可串行化调度

具有什么性质的调度是可串行化的调度？如何判断一个调度是否可串行化？本节给出判断可串行化调度的充分条件。

首先介绍冲突操作的概念。

冲突操作是指不同的事务对同一个数据的读写操作和写写操作：

$R_i(x)$ 与 $W_j(x)$ /* 事务 T_i 读 x , T_j 写 x , 其中 $i \neq j$ */

$W_i(x)$ 与 $W_j(x)$ /* 事务 T_i 写 x , T_j 写 x , 其中 $i \neq j$ */

其他操作是不冲突操作。

不同事务的冲突操作和同一事务的两个操作是不能互换 (swap) 的。对于 $R_i(x)$ 与 $W_j(x)$, 若改变二者的次序, 则事务 T_i 看到的数据库状态就发生了改变, 自然会影响到事务 T_i 后面的行为。对于 $W_i(x)$ 与 $W_j(x)$, 改变二者的次序也会影响数据库的状态, x 的值由等于 T_j 的结果变成了等于 T_i 的结果。

一个调度 Sc 在保证冲突操作的次序不变的情况下, 通过交换两个事务不冲突操作的次序得到另一个调度 Sc' , 如果 Sc' 是串行的, 称调度 Sc 为冲突可串行化的调度。若一个调度是冲突可串行化的, 则一定是可串行化的调度。因此, 可以用这种方法来判断一个调度是否为冲突可串行化的。

[例 12.3] 今有调度 $Sc_1 = R_1(A)W_1(A)R_2(A)W_2(A)R_1(B)W_1(B)R_2(B)W_2(B)$, 假设 T_1 和 T_2 均正常提交。

可以把 $W_2(A)$ 与 $R_1(B)W_1(B)$ 交换, 得到

$R_1(A)W_1(A)R_2(A)R_1(B)W_1(B)W_2(A)R_2(B)W_2(B)$

再把 $R_2(A)$ 与 $R_1(B)W_1(B)$ 交换

$Sc_2 = R_1(A)W_1(A)R_1(B)W_1(B)R_2(A)W_2(A)R_2(B)W_2(B)$

Sc_2 等价于一个串行调度 T_1, T_2 。所以 Sc_1 为冲突可串行化的调度 (详细讲解参见二维码内容)。



微视频：冲突可
串行化判断示例

应该指出的是, 冲突可串行化调度是可串行化调度的充分条件, 不是必要条件。还有不满足冲突可串行化条件的可串行化调度。

[例 12.4] 有三个事务 $T_1 = W_1(Y)W_1(X)$, $T_2 = W_2(Y)W_2(X)$, $T_3 = W_3(X)$ 。

调度 $L_1 = W_1(Y)W_1(X)W_2(Y)W_2(X)W_3(X)$ 是一个串行调度。

调度 $L_2 = W_1(Y)W_2(Y)W_2(X)W_1(X)W_3(X)$ 不满足冲突可串行化。但是调度 L_2 是可串行化的, 因为 L_2 执行的结果与调度 L_1 相同, Y 的值都等于 T_2 的值, X 的值都等于 T_3 的值。

前面已经讲到, 商用数据库管理系统的并发控制一般采用封锁的方法来实现, 那么如何使封锁机制能够产生可串行化调度呢? 下面将要讲解的两段锁协议就可以实现可串行化调度。

12.7 两段锁协议

为了保证并发调度的正确性, 数据库管理系统的并发控制机制必须提供一定的手段来保证调度是可串行化的。目前数据库管理系统普遍采用两段封锁^① (two-phase lock, 2PL, 简称两段

① 《计算机科学技术名词》第3版中译为“两阶段锁”。